A Graphical Workbench and a Verified Bridge for the Daisy Model: From a Question in Plain Words to a Decision Layer, in One Offline Application

Naziru Halilu[1,2,*]

[1] Agricultural University of Athens, Athens, Greece

[2] Public University of Navarre, Pamplona, Spain

ORCID: 0009-0006-2324-9599

[*] Correspondence: halilunaziru73@gmail.com

Abstract

Daisy is a process-based model of the soil–plant–atmosphere system that has been developed and validated for more than three decades, but its use requires text set-up files, a command-line program and plain-text logs, and its results reach a decision only through hand-made steps in which the origin of a number is lost. This article describes and tests a graphical workbench for Daisy and a bridge from its output to decision layers, both parts of the offline SWAFL Desktop 1.1.0. The workbench is a guided path of nine steps with a planner that turns a request in plain words into a design (an offline reader, with an optional online assistant whose answer is checked against the catalogue of what Daisy can vary and repaired or refused), a decision base read from the Daisy reference manual (107 chapters, 999 sections), 17 prepared experiments, a process map of 14 processes and 76 components, a one-dimensional Richards soil column, response curves, three optimisers for calibration, and a citation and the raw Daisy files kept with every figure and run. The bridge has five parts and one rule: a decision layer draws a Daisy result only from a Daisy run, and otherwise says that there is none. Each part was tested against an independent reference. The planner returned the expected factors, station and soil for all 8 test questions, and its check dropped an unknown factor and limited out-of-range levels. Compact designs of 6 to 72 runs recover the marginal curves of the 10 190-run study to a median of 13 % of their range at 1/25 of the Daisy time. The soil column conserves mass to better than 0.001 mm per day and agrees with a scipy reference to 0.3 and 0.9 % of the drainage. The log parsers agree with an independent reader to 3×10^{-15} and the chemical budget of a tracer log closes to 0.02 % of the amount applied. The test of the denitrification function against 792 stored Daisy outputs exposed a defect (median error 40 %) that is corrected, after which the agreement is 3×10^{-15}. Three routes from Daisy to a decision (ensemble, meta-model, live run) differ by seven orders of magnitude in time. Daisy itself was not run in these tests, the online assistant and the online data services were not reached, and no field data are used.

Keywords. Daisy model; graphical user interface; question planner; model coupling; decision support; experiment design; Richards equation; model calibration; denitrification; verification; provenance; open science

1. Introduction

Process-based models of the soil–plant–atmosphere system are the main tools for asking what would happen to the water, the crop and the nitrogen of a field if the management or the climate changed. Daisy is one of them. It couples the water and heat flow in the soil, the soil organic matter and the nitrogen, the crop and the surface, and it has been described, calibrated and tested in a long series of papers (Abrahamsen and Hansen, 2000; Hansen et al., 1990, 2012; Holbak et al., 2026). Comparable models are used for crops and irrigation (Jones et al., 2003; Keating et al., 2003; Steduto et al., 2009) and for the flow of water and solutes (Šimůnek et al., 2008, 2016). What these models have in common is that a simulation is specified in a text file, run by a program, and read from files. That is natural for a model developer. It is a barrier for the person who has a question about a field: how much of the water that a mulch saves is lost to drainage, which trigger suction suits a sandy soil, what a smaller nitrogen dose does to the nitrate that leaves the root zone.

The barrier has a second part. A set of simulations that answers a question (a factorial design over the trigger, the sensor depth and the rain, say) is itself a piece of work that is rarely recorded with the answer. A published ensemble of 10 190 Daisy runs in 17 experiments (Halilu, 2026) makes the design of the study explicit, but a reader who wants to repeat one experiment on a laptop, with another soil or another weather, has no easy way to do so.

A simulation model is a function from a description of a field to a set of time series. A decision is a choice between management options for a particular field. The distance between the two is crossed by operations that are rarely described or tested: the reading of the output, its reduction to a few numbers per season, the matching of the simulated soil to the soil of the field, the choice of the output that answers the question, and the presentation of the answer next to its alternatives. When a decision tool is built on a process model, these operations are the tool.

Two strategies are common. In the first, the model is run offline over a design of scenarios and a fast statistical model (a meta-model, emulator or surrogate) is fitted to the results (Kennedy and O'Hagan, 2001; Razavi et al., 2012; Sacks et al., 1989); the decision tool then evaluates the emulator. This is fast and needs no model installation, but it can only answer for the conditions of the design. In the second, the tool runs the model on demand for the user's own conditions, which is general but slow and needs the model. Both have a place in a tool for irrigation planning, and the tool described here uses both: a meta-model of a published ensemble of 10 190 Daisy runs

(Halilu, 2026) is Engine 1 of the suite (Paper 2), and the graphical workbench for Daisy described here runs Daisy for the user and hands the results on.

This article describes both parts together because they are one chain: the workbench produces the Daisy runs, the bridge hands them to the decision layers, and the rule of origin is what keeps the chain checkable. We describe (i) how the workbench is organised and why, (ii) how a request in words becomes a design and how every figure and run keeps its origin, (iii) how the experiments of the published ensemble are made available as prepared designs and what is lost when they are reduced to a size that a laptop can run, (iv) the tools that compute inside the workbench, (v) the components of the bridge and the rule by which the decision layers use them, and (vi) the checks that were made of each part. The checks are part of the contribution: a graphical tool that shows a number does not make the number right, and a bridge that mistranslates a log is worse than no bridge because it produces numbers that look like Daisy output.

Contributions. (1) A guided path from site data to a decision, in which each step is one tab and each tab states what it is for. (2) A planner that turns a request in plain words into a design, with an online assistant that is held to the catalogue of what Daisy can vary. (3) A decision base read from the Daisy reference manual, with the section of the manual that each choice comes from. (4) Seventeen prepared experiments, with their compact designs, their run-time estimates and the study results next to them. (5) A soil-column solver, response curves and a three-optimiser calibration tool that run without Daisy. (6) A bridge with five small parts and a single rule that keeps the origin of every Daisy number visible, and a comparison of the three routes from Daisy to a decision. (7) A verification of every computing part against files or solvers that were not made by the workbench, with the defect of the shared denitrification function that the verification found.

What is new. Daisy is a mature process model that is run from text files, and decision tools that use it normally receive its results by hand. The new element is the chain, under one rule of origin: a request in plain words is turned into a design that is checked against the catalogue of the model (offline, or with an online assistant whose answer cannot add a factor or a value that the catalogue does not contain), the design is run or taken from the prepared experiments of a published ensemble with a stated cost of reduction, every figure carries the citation of its weather, soil and run and every run keeps the files that Daisy wrote exactly as written, and the results are handed to decision layers that show a Daisy result only from a Daisy run. Around the chain are an inspectable map of the processes and components, a soil-column solver, response curves

and a three-optimiser calibration tool that work before Daisy is started (no claim is made about other tools). Each computing part is tested against an independent reference, and the test of the denitrification function found a defect that had been in the shared code, which is corrected.

2. The workbench

2.1 Design principles

Four principles shaped the workbench. *Questions before syntax*: the first thing a user does is to state a question, either by choosing a scenario or a prepared experiment or by writing a request, and the set-up file is the result and not the starting point. *Everything that runs is shown first*: a design lists its runs, their settings and an estimate of the time before anything is run. *One source of data*: the weather and the soil of the site are entered once and every tab uses them. *No hidden steps between a result and a decision*: the last step of the path hands the results of the runs to the layers of the decision-support suite (the bridge below).

2.2 Structure

The workbench is a window of the application with six groups of tabs (Fig. 1). The guided path has nine steps: weather and soil of the site; the library of soils, crops and fertilisers; scenarios; experiments; the process map; log plots; mass balance; run comparison; and the hand-over to the SWAFL layers. A second row holds the tools of model development, which are outside the path: the set-up editor for `.dai` files, the soil column, the response curves, calibration, and the documentation. The documentation tab holds 107 reference topics, 27 combinations of factors with a recommended design, and a manual of the model. The bundled sample data are seven weather records (Taastrup, Foulum, Jyndevad, Bologna, Jadwisin, Valecov, Bratislava), ten annual or daily logs of the Daisy project, and the horizon, crop, fertiliser and action library read from its `.dai` files. Daisy 7.1.14 for Windows is bundled with the application and is the program that runs the set-ups; the workbench writes the set-up and reads the logs back.

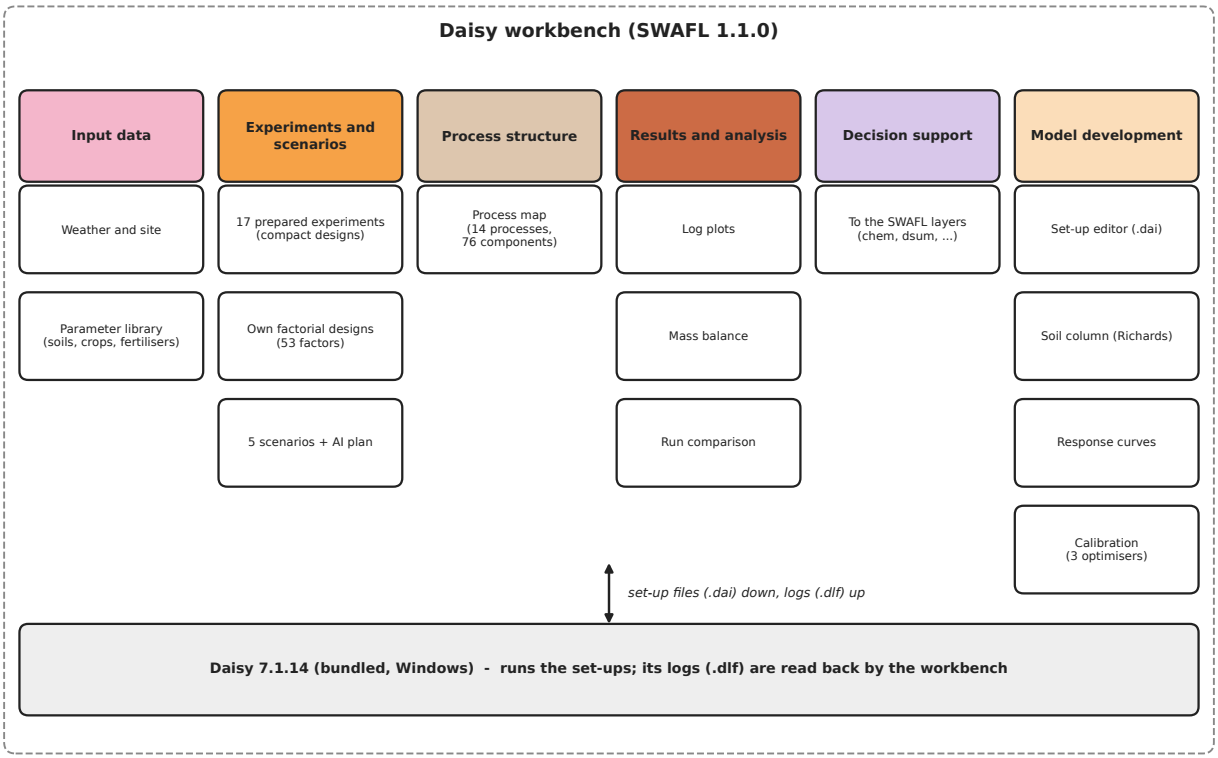


Figure 1: Organisation of the Daisy workbench: six groups of tabs and the Daisy program that runs the set-ups. The guided path runs from the input data to the hand-over to decision support; the model-development group is outside the path.

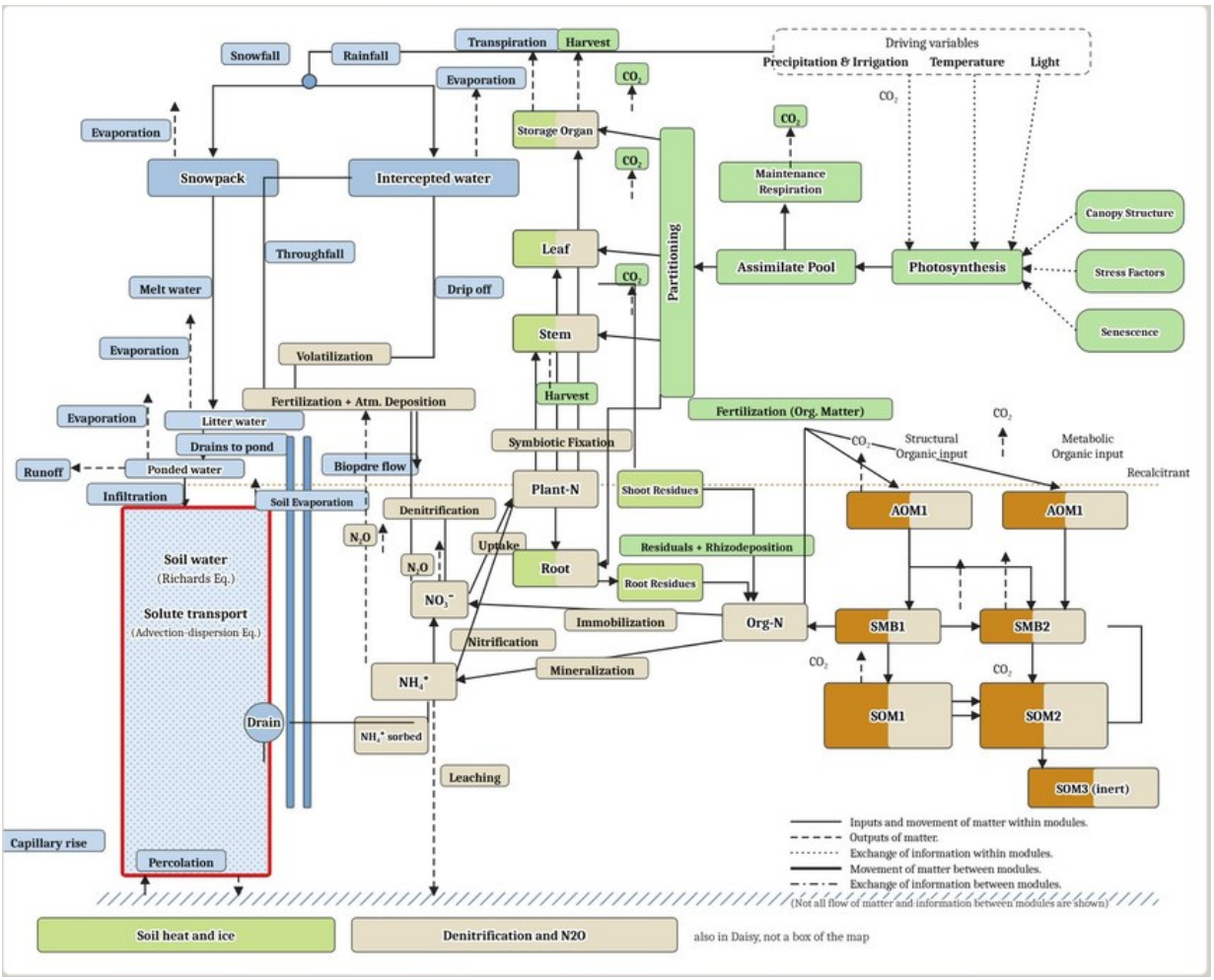


Figure 2: The process map of Daisy. Each process is a button that opens the figures it produces from a run.

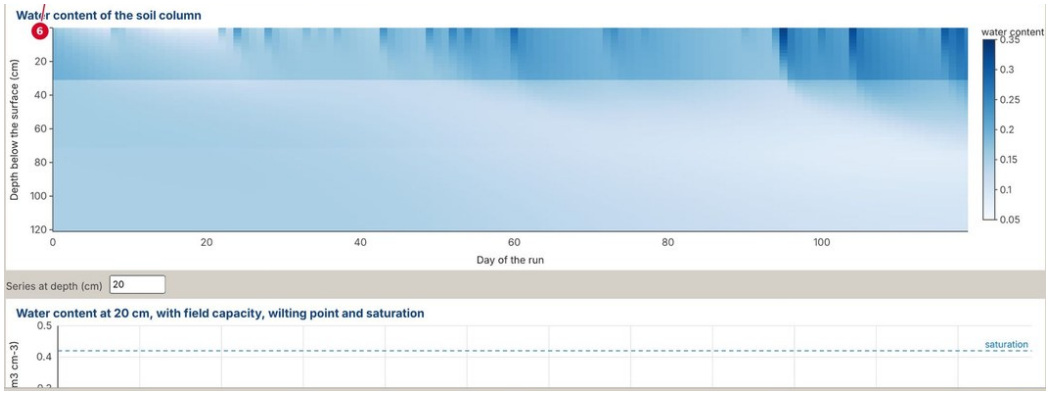


Figure 3: Water content of a soil column through time, computed by the Richards solver of the workbench.

2.3 Experiments and scenarios

A scenario is a prepared question with a recommended set of runs: irrigation strategy, nitrogen fertilisation and leaching, climate, the fate of a chemical, and the effect of the soil. An experiment

is a factorial design that the user builds from factors. The 53 factors cover the weather (rain scale, temperature shift), the soil (a library horizon or a texture class), irrigation (method, trigger suction, sensor depth, dose, window), nitrogen (rate), the crop (rooting depth), tillage and the parameters of soil structure. For a design the workbench lists every run, with its settings, before it is started, shows the number of runs against a limit and the estimated time, and writes the set-ups. For the 17 experiments of the published ensemble a prepared design is offered, each with its own figures and its own folder of results. The same factor definitions are used to read the results of an ensemble (the bundled ensemble file with the 10 190 runs) as marginal means, grids, sensitivities and Pareto fronts.

3. Question, reference and provenance

3.1 The planner

The user writes what is to be found out ("how does the irrigation trigger of -100 to -800 cm change drainage on a sandy loam in Bologna over 10 years?") and a plan comes back: the factors and their levels, the design (factorial, one at a time or paired), the weather station, the soil, the years, the responses to look at and the questions that remain, each with its options. Two engines produce the same plan. The built-in reader works offline and anywhere: it recognises the soils, stations, factors and responses of the catalogue by their names and meanings, reads quantities with units ("10, 30 and 60 cm", "20 %", "2 degrees"), ranges with steps and numbers of years, and keeps every number inside the range of its factor. The online assistant is a language model that is given the catalogue and must answer with the plan in the same structure. Its answer is never used as it stands: a factor that is not in the catalogue is left out and noted, levels are limited to the range of the factor, a soil, station or response that is not in the catalogue is removed, and the built-in reader is used if the answer fails. The assistant therefore cannot make the workbench ask Daisy for something that Daisy cannot vary.

3.2 The decision base from the reference manual

The choices of an experiment are tied to the Daisy reference manual (version 7.1.14), which is embedded as its 107 chapters with the title, introduction and, for each model, the name, first line of the description and the names of the parameters (999 sections). For each factor the base records the section of the manual that the choice comes from, and it proposes the combinations

worth running first for the weather and the soil of the user; no experiment of an earlier study is used to make the proposal. A table of 32 parameters of the reference records, for each, the manual default and whether a change altered the yield, water or nitrogen outputs in a test of Daisy 7.1.14 on the farm of the application (the parameter was written into a run at the manual default and at half and double that value); 12 of the 32 did. This test is recorded in the source of the workbench and was not repeated in this study, in which Daisy could not be run.

3.3 Provenance of figures and runs

Every figure that the workbench draws carries a citation line that names Daisy 7.1.14 (Hansen et al., 2012; Holbak et al., 2026) and the weather, the soil and the options of the run that made it, below the figure and burnt into the picture when it is saved, and the list of references of the workbench is built from the same source. Every run keeps all the files that Daisy wrote (the `.dlf` logs, the `.csv` files, `daisy.log`) exactly as written and never changed by the application, each with a download button, a `.zip` of the outputs, a `.zip` that also holds the set-up, and the folder on the computer where the desktop application keeps the run. The summary of a run that the decision layers read is made by the same code for the bundled results and for the run just made, so the two are comparable.

3.4 Calculators that run without Daisy

The workbench computes in the browser the van Genuchten and Mualem hydraulics of the Daisy horizons and a one-dimensional Richards solver (the soil column below), the Daisy denitrification function, and the methods of the Daisy optimisation tools for calibration: a sequential search, a covariance-matrix evolution strategy with rank-one and rank-μ updates (Hansen and Ostermeier, 2001) and Bayesian optimisation (Mockus et al., 1978). It also reads the open data of the Danish Meteorological Institute (with the user's own key) and finds the closest station. The weather of the area of any place on Earth can be fetched from the Open-Meteo archive (ERA5 based) and written as a Daisy weather file, and the user's own files are read as CSV, Excel, DWF, DLF or DAI. Weather is therefore a choice made for every project and every run, and not a record fixed in the program.

4. The bridge

4.1 Parts

Figure 4 shows the bridge. A scenario or an experiment of the workbench is turned into a Daisy set-up and run by Daisy 7.1.14. Daisy writes its logs as tab-separated files (`.dlf`) with a header, a row of names, a row of units and one row per output interval. The bridge has five parts.

Log parser (`DaisyIO`, `DaisySum.parseDlf`).
Reads a log into a list of names and rows. Numeric columns become numbers; the crop name stays text. The first row of an annual log is written at the start of the simulation and is not a season; the summariser drops it.

Seasonal summariser (`DaisySum.summarise`).
Reduces the logs of a run to one record with a value per season: harvest of the crop (dry matter of the storage organ, water stress, nitrogen stress), precipitation, irrigation, actual and potential evapotranspiration, runoff, percolation below 1 m (matrix and biopore flow), and the nitrogen terms of Eq. (1). For a run of a chemical it adds the budget of Eq. (2).

Soil map (`SOILMAP`).
Maps the soil of a Daisy run to the soil of the suite (for example `usda:siltloam` to Silt Loam, `S07`), so that the other layers show the same soil. Eight soils are mapped (sand, loamy sand, sandy loam, loam, silt loam, sandy clay loam, clay loam, silty clay loam); the two coarse soils that are special to the study (Coarse Loam, Coarse Sand) and an own soil table have no counterpart, and the layers then keep the soil of the project.

Question map (`MATCH`).
For each of five questions (irrigation, nitrogen, climate, a chemical, the soil) lists the layers that answer it with a sentence on what each shows (Table 1). The hand-over tab of the workbench opens the first of them.

Live object (`DAISYLIVE`).
Holds, in the engine of the suite, the label of the weather and soil of the run, the summarised runs, their settings (irrigation, nitrogen factor, rain factor, temperature shift) and the mapped soil. The decision layers read only this object.

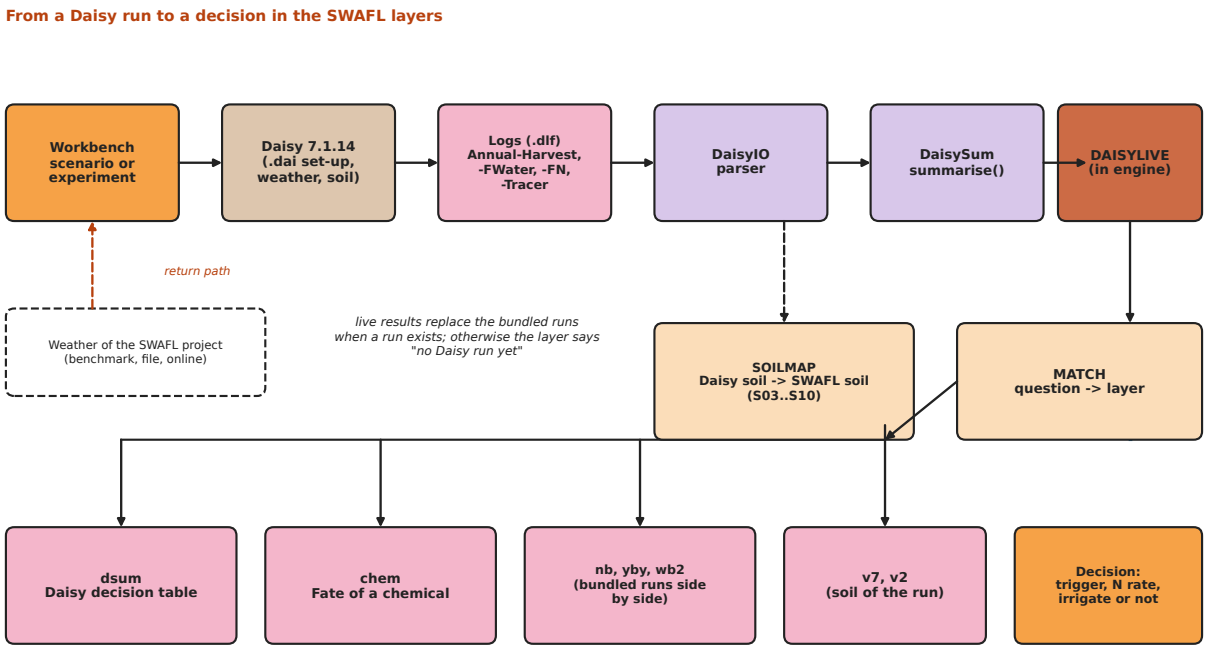


Figure 4: The bridge from a Daisy run to a decision. Solid arrows are the path of a run; the dashed arrow at the left is the return path of the weather of the project to Daisy.

Table 1: Questions of the workbench and the layers that answer them.

Question	Layers
Irrigation: does it pay?	Daisy decision table (`dsum`); reliability over 20 seasons (`yby`); full water balance (`wb2`); sensor and trigger (`v1`)
Nitrogen: how much fertiliser?	decision table; nitrogen budget (`nb`)
Climate: warmer, drier, wetter	decision table; reliability over 20 seasons
A chemical: sorption and decay	fate of a chemical (`chem`)
The soil: how much does texture matter?	soil hydraulic drivers (`v7`); profile evolution (`v2`); decision table
Own set-up	decision table (own runs next to the others when the names match)

4.2 What the summariser computes

For season s the nitrogen terms are sums of columns of the annual nitrogen log (kg N ha^{-1}):

$$\begin{aligned} N_s^{\text{fert}} &= N_{\text{surf}} + N_{\text{soil}} + N_{\text{org}}, \quad N_s^{\text{up}} = N_{\text{uptake}}, \\ N_s^{\text{den}} &= N_{\text{denit}} + N_{\text{N}_2\text{O nitrif}}, \qquad N_s^{L} = N_{\text{matrix leach}} + N_{\text{biopore leach}} \ \ (\text{at } 100 \text{ cm}), \end{aligned} \tag{1}$$

together with the mineralisation, the deposition and the volatilisation. The chemical budget uses the annual tracer log (kg ha^{-1}; the log is in g):

$$\Delta M_s = A_s - U_s - L_s - R_s - X_s, \qquad L_s = L_{\text{matrix}} + L_{\text{biopore}} + L_{\text{soil drain}} + L_{\text{biopore drain}}, \quad (2)$$

where A is the spray, U the crop uptake, R the runoff, X the decay (soil, surface and litter), and ΔM the change of the amount in the soil from one season to the next. A depth profile of the concentration is added from the daily concentration log. The weather and soil of the run are described by a one-line label, shown at the top of every Daisy layer ("Daisy 7.1.14, run in the Daisy workbench: weather Taastrup, Denmark; soil loam; years 1979 to 2000").

4.3 The rule of origin

A decision layer that shows a Daisy quantity draws it from a live run or not at all. Before a run, the two layers of Daisy output (the decision table and the fate of a chemical) show the sentence "no Daisy run yet", say what is needed to fill them, and offer a button to the scenarios of the workbench; they do not substitute bundled or estimated numbers. The other layers of the suite, which are driven by Engine 1 and Engine 2, say in their own header that they show the meta-model and the daily model. The rule is a small feature, but it settles a question that every coupled tool has to answer: when a number is on the screen, is it from the model it is labelled with? In a test with the application opened afresh (`p5_empty_state.js`) both layers showed this sentence and no figure, and the engine held no live object.

5. Methods of the workbench checks

5.1 Design size and fidelity

Each experiment of the study was a full-factorial grid with 10 levels per factor, which gave 200 to 1 000 runs per experiment. A laptop cannot repeat that in a working session, so each prepared experiment is offered as a compact design with 3 to 6 levels per factor (Table 3). The size of every compact design was obtained from the workbench code, and the time of the study from the recorded wall-clock time of every run in the archive of the study (`run_seconds`). The time of a compact design was estimated as its number of runs times the median run time of its experiment.

To ask what the reduction costs, the marginal mean of five responses (yield, irrigation,

drainage at 1 m, irrigation efficiency, nitrate leached) was computed for each numeric factor of each experiment from the full ensemble, and then rebuilt by linear interpolation between the marginal means at the levels of the compact design (the nearest study level to each compact level; the suction of the trigger interpolated on a logarithmic scale). The root-mean-square difference between the rebuilt and the full curve, as a percentage of the range of the full curve, measures the information that the compact design cannot see by itself. It is a stricter test than the practical use, in which the user chooses levels where the response is steep, but it has no free parameter.

5.2 Soil column

The soil column solves the one-dimensional Richards equation for vertical flow in the mixed form

$$\frac{\partial \theta}{\partial t} = -\frac{\partial q}{\partial z} - S(h,z), \qquad q = K(h)\left(1 - \frac{\partial h}{\partial z}\right), \tag{3}$$

where the depth z is positive downward, q is the downward flux and S the root sink. The retention curve and the conductivity are those of van Genuchten (1980) and Mualem (1976),

$$S_e = \left[1 + (\alpha|h|)^n\right]^{-m}, \quad m = 1 - \tfrac{1}{n}, \quad \theta = \theta_r + (\theta_s - \theta_r) S_e, \quad K = K_s S_e^{\ell}\left[1 - \left(1 - S_e^{1/m}\right)^m\right]^2, \tag{4}$$

with $\ell = 0.5$ unless the horizon gives another value. The root sink is the potential transpiration distributed over a linearly decreasing root density and reduced by the stress function of Feddes et al. (1978), which is 1 between suctions of 25 and 500 cm, falls linearly to 0 at 15 000 cm and rises linearly from 0 at the surface to 1 at 25 cm. The equation is discretised by finite volumes on nodes with implicit time steps and Picard iteration with the mass-conserving capacity term of Celia et al. (1990); the time step grows when the iteration converges in five iterations or fewer and shrinks when it takes more than ten. The upper boundary takes the rain less the evaporation demand as a flux until the surface node would be positive, when it becomes a ponded boundary ($h = 0$) and the excess runs off; when the soil is air-dry ($h < -10^4$ cm) the evaporation is reduced to what the soil can give. The lower boundary is free drainage, a water table at a given depth, or a closed bottom. The tool returns the daily infiltration, runoff, evaporation, transpiration, drainage and storage and the daily mass-balance error $\epsilon = \Delta S - (I - E - T - D)$ (in mm).

To test it, three soils were set up from van Genuchten parameters of loam-type classes (Table 2), as a 100 cm column at a suction of 300 cm, with 200 mm of rain on day 1 and five dry days,

free drainage, node spacings of 4, 2, 1 and 0.5 cm. For sand and loam the rain rate is below the saturated conductivity, so the upper boundary is a flux; an independent reference was calculated with the method of lines (scipy BDF with a banded Jacobian, 0.5 cm nodes, relative tolerance 10^{-6}) and the same flux. The silty clay loam has a saturated conductivity of $0.07\,\mathrm{cm\,h^{-1}}$, so the surface ponds; for it only the convergence with the node spacing and the run time are reported.

Table 2: Soils used for the test of the soil column (van Genuchten parameters, $\ell = 0.5$).

Soil	α ($\mathrm{cm^{-1}}$)	n	K_s ($\mathrm{cm\,h^{-1}}$)	θ_s	θ_r
Sand	0.145	2.68	29.7	0.43	0.045
Loam	0.036	1.56	1.04	0.43	0.078
Silty clay loam	0.010	1.23	0.07	0.50	0.089

5.3 Response curves: denitrification

The response-curve tool draws the denitrification function that Daisy uses by default in the form given in its Python extensions. The water-filled pore space $w = \theta/\theta_s$ gives a piecewise-linear moisture factor $f_w(w)$ (zero at $w = 0$, 0.01 at 0.4, 0.07 at 0.5, 0.17 at 0.57, 0.32 at 0.64, 0.51 at 0.71, 0.66 at 0.76, 0.79 at 0.81, 0.9 at 0.85 and 1 from 0.9). The potential rate is $f_T\,[\alpha_F C_F + \alpha(C - C_F)]$ with the fast and the total carbon dioxide production C_F and C and the factors $\alpha_F = 0.0530$ and $\alpha = 10^{-4}$. The actual rate is limited by the nitrate, $D = \min(f_w r_S, K_d N) + \min(f_w r_F, K_d N)$ with $K_d = 0.0208\,\mathrm{h^{-1}}$. The loss is divided between the two gases by a ratio r,

$$D_{\mathrm{N_2O}} = \frac{D}{1+r}, \qquad D_{\mathrm{N_2}} = \frac{D}{1+1/r}, \qquad r = \min(r_{\mathrm{NO_3}}, r_{\mathrm{CO_2}})\, r_w, \tag{5}$$

where $r_{\mathrm{NO_3}} = 25\left[1 - \frac{1}{2} - \frac{1}{\pi}\arctan\big(0.01\pi(N_m - 190)\big)\right]$ depends on the nitrate concentration N_m (mg per kg of soil), $r_{\mathrm{CO_2}} = 13 + \frac{30.78}{\pi}\arctan\big(0.07\pi(C_m - 13)\big)$ on the carbon dioxide production over a 30 cm layer, C_m, and $r_w = 1.4 \times 13^{-17/13^{2.2w}}$ on the water-filled pore space. The Daisy project stores 792 inputs of this function with the outputs that a Daisy run gave for them. The workbench function was compared with all 792.

5.4 Calibration

The calibration tool minimises a loss (mean squared error, mean absolute error or sum of squared deviations) between a model output and a target over up to a few parameters with bounds. Three

optimisers are offered, after the choices of the Daisy optimisation project: a sequential search in which each parameter takes a few values in turn and the single best change is kept; the covariance-matrix-adaptation evolution strategy (Hansen and Ostermeier, 2001); and a Bayesian search with a Gaussian-process surrogate and expected improvement (Mockus et al., 1978). To test them, the three parameters α_F, K_d and α of the denitrification function were estimated from the 792 stored outputs (the loss is the mean squared error of the logarithm of the N_2 loss), starting from values 3 times too high, with a budget of 150 evaluations and five seeds. The correct values are known, so the recovery can be measured as the error in decades.

5.5 Reproducibility

All checks are scripts in `Manuscripts/reproducibility/p456` (Appendix A). The design sizes are obtained by loading the workbench code in Node.js; the ensemble analysis is in Python.

5.6 Planner test

Eight questions in plain words were written, each with the factors, the station and the soil that the plan has to contain (for example a trigger range on a sandy loam in Bologna, a nitrogen cut in Foulum, rain at plus and minus 20 % on a loamy sand at Jyndevad). The offline reader was run on each (`p7_planner_checks.js`); the guard of the online route was run on an invented answer that holds an unknown factor, a rain scale of 0.1 and 9, an unknown station and soil and an unknown response.

6. Methods of the bridge tests

Parsers. The six Daisy logs bundled with the suite (annual field nitrogen, two runs; annual tracer; daily water flux; the nitrogen log of a calibration example; a soil chemical log with 11 665 rows) were read by the parser of the bridge and, independently, by `pandas`. For every numeric column the sum was compared; the largest relative difference is reported.

Nitrogen summaries. The two annual nitrogen logs hold ten seasons of a rotation, with the same fertiliser and different set-ups (runs 3b and 4b). For each season the summariser was compared with the sum of the corresponding log columns of Eq. (1), and the closure column of the log (Daisy’s own error term) was read.

Chemical budget. The annual tracer log was passed through the summariser with chemistry, and the residual of Eq. (2) was computed for each of the ten seasons from the summarised amounts.

Denitrification. The function that the workbench shares with Daisy was compared with the 792 inputs and stored outputs of the Daisy denitrification test (the function and its defect are described with the workbench checks).

Time and agreement of the three routes. The Daisy time of the ensemble and of its experiments was summed from the recorded run times (`run_seconds`); the time of Engine 1 and its agreement with Daisy on held-out runs are those of Paper 2.

7. Results of the workbench checks

7.1 Prepared designs and what the reduction costs

Fourteen of the 17 experiments are reproduced as compact designs of 6 to 72 runs (Table 3); the three that are not (mulching, rain intensity and moisture, mulch and rain) are available through their published runs. The 368 runs of the compact designs stand for 7 990 runs of the study, 4.6 %, and for 361 hours of Daisy time in the study against about 15 hours (a factor of 25). The whole study took 17.5 days of Daisy wall-clock time on the machines that ran it (mean 148 s, median 145 s per run), which is the reason why a prepared design cannot be the full design.

Table 3: The 17 experiments of the study and their prepared designs in the workbench. Hours are the sum of the Daisy wall-clock run times of the study (`run_seconds`) and the estimate for the compact design (runs × the median run time of the experiment). The experiments marked – are not offered as designs; their published runs are bundled.

Experiment	Question	Study runs	Study (h)	Compact runs	Compact (h)
E0	Baselines	10	0.6	9	0.6
E1	Dose and duration	200	11.0	17	0.9
E2	Trigger pressure	200	7.7	21	0.8
E3	Sensor depth	200	8.1	21	0.8
E4	Methods	100	4.1	6	0.2
E5	Soil structure dynamics (SHP)	300	15.8	72	3.5
E6	Mediterranean pollution	210	6.9	30	0.8
E7	Mulching	200	8.9	–	–
E8	Rain intensity and moisture	1000	1.7	–	–
E9	Soil moisture and rain	1000	2.0	40	0.1
E10	Amount, timing and soil	1000	70.4	20	0.7
E11	Temperature, evaporation, rain	1000	42.0	16	0.7
E12	Sensor, trigger, rain	1000	42.9	30	1.2
E13	N, water, soil	1000	57.4	32	1.9
E14	Mulch and rain	1000	48.0	–	–
E15	Method, depth, amount	1000	43.6	30	1.3
E16	Edge cases	770	48.2	24	1.2
Total		10190	419	368	14.7

The marginal curves are recovered with a median error of 13.2 % of their range over the 51 combinations of experiment, factor and response (90th percentile 29.9 %, largest 36.1 %; Fig. 5a). The error is similar for the five responses (median 11.0 to 16.9 %). It is smallest where the compact design has as many levels as the response needs (sensor depth, E3, 4.9 %; trigger suction in E2, 7.3 %) and largest where three levels must describe a response that bends (the dose in E10, 30 %; the sensor depth in the three-level E12, 21 %). The result says that the compact designs are for orientation and for the comparison of options, and that a curve that matters for a decision should be run with more levels where it bends; the workbench makes this a matter of one setting.

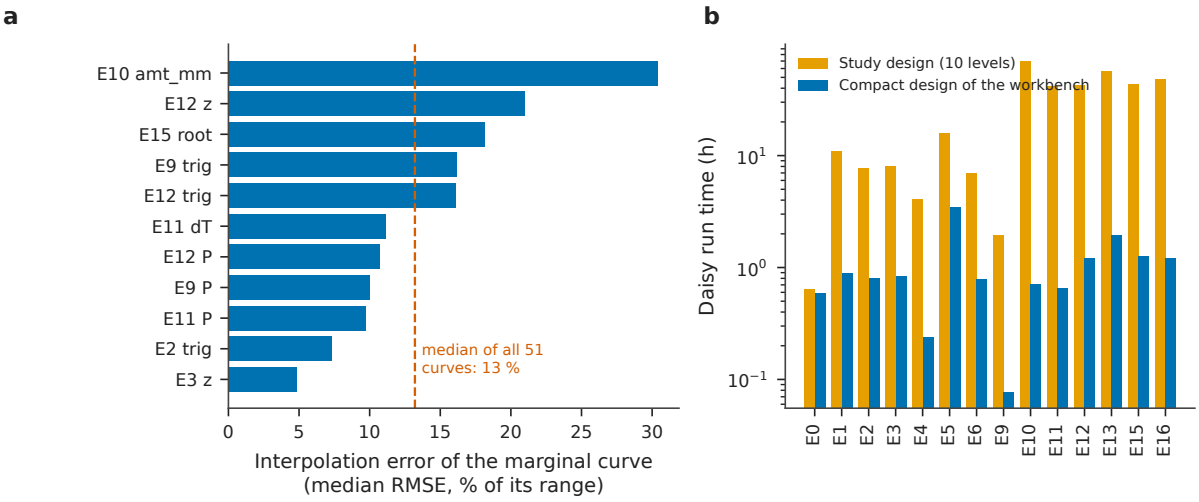


Figure 5: (a) Error of the marginal curves rebuilt from the levels of the compact designs (median over five responses, per experiment and factor). (b) Daisy run time of the study designs and of the compact designs, by experiment (logarithmic scale).

7.2 The soil column

In sand and loam the solver conserves mass: the largest daily error is below 0.001 mm at every node spacing. The drainage below 1 m after 6 days differs from the reference by 0.46 mm in sand (152.14 against 152.60 mm; 0.3 %) and by 0.50 mm in loam (54.62 against 55.12 mm; 0.9 %), and the final storage by 0.42 and 0.50 mm, 0.45 and 0.16 % of the store (Table 4, Fig. 6a,b). The result hardly depends on the node spacing: the drainage varies by 0.03 mm in sand and 0.05 mm in loam between 4 and 0.5 cm. The solver is therefore not sensitive to the default spacing of 2 cm in these two soils, and the remaining difference to the reference is a difference of method (a time-step control against a high-order integrator) that is small against any parameter uncertainty.

Table 4: The Richards solver of the workbench against an independent reference (method of lines, scipy BDF, 0.5 cm nodes). A 100 cm column, initial suction 300 cm, 200 mm of rain on day 1, five dry days, free drainage. D is the drainage below 1 m and S the final storage in mm, for the reference (ref) and the workbench (wb, 0.5 cm nodes); the spread (mm) is the range over node spacings of 4, 2, 1 and 0.5 cm.

Soil	D_{ref}	D_{wb}	Spread	S_{ref}	S_{wb}
Sand	152.60	152.14	0.03	93.12	93.54
Loam	55.12	54.62	0.05	314.94	315.44

The silty clay loam shows the limit. With the surface ponded, the infiltration of the 200 mm depends on the node spacing (24.4, 22.5 and 21.8 mm at 4, 2 and 1 cm), the mass-balance error rises to 0.05 to 0.07 mm per day, and the run time at 1 cm is 182 s against 21 s at 2 and 4 cm, with 1.3 million Picard iterations (Fig. 6c). The infiltration into a dry, fine-textured soil has a

sharp front that the node spacing of 1 to 4 cm cannot resolve, and the surface condition switches between flux and ponding. For such a soil the solver gives the right order of magnitude but not a converged number; the tool shows its mass-balance error and the iteration count so that this can be seen.

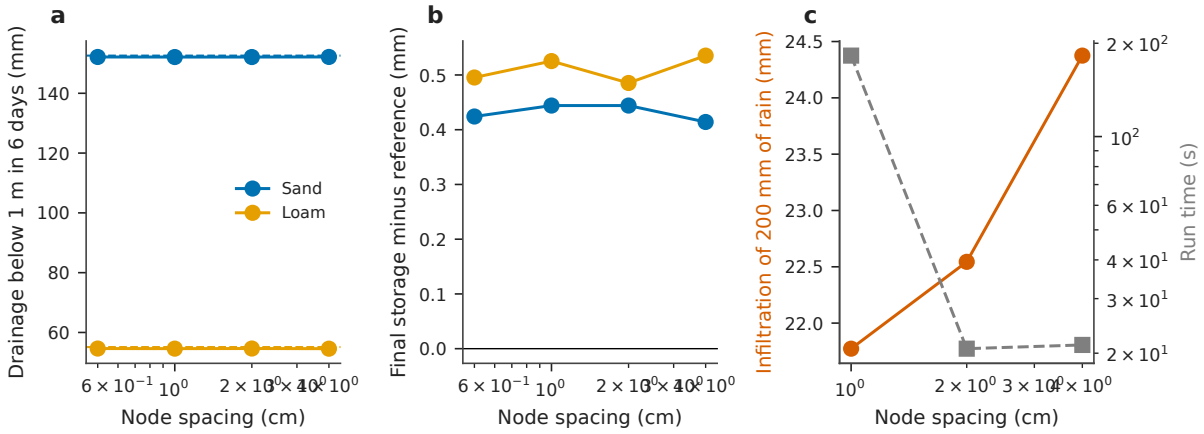


Figure 6: Soil column. (a) Drainage below 1 m with the node spacing; dashed lines are the reference. (b) Final storage minus the reference. (c) Infiltration of 200 mm of rain into a ponded silty clay loam (left axis) and run time (right axis), showing the lack of convergence.

7.3 The denitrification function and a defect that the check found

The first comparison with the 792 stored outputs showed that the function of release 1.1.0 gave 60 % of the stored N_2 and N_2O loss (median relative error 40 % for both gases; Fig. 7). The Python extension of the Daisy project, which the workbench function had been ported from, gives the same 60 % against the same file, so the port was faithful to the extension; the extension applies the correction for the pressure head ($\times 0.6/f_h$) to the carbon dioxide production, whereas the stored Daisy outputs were produced from inputs that had already been corrected. Taking the inputs as the values that Daisy passes to the function, and using the uncorrected value $Cf_h/0.6$ only in the regulation of the N_2O fraction (Eq. (5)), reproduces all 792 rows of N_2 and of N_2O with a largest relative difference of 2.9×10^{-15} for N_2 and 1.7×10^{-15} for N_2O. The corrected form is the default of release 1.1.0, and the form of the extension is kept as an option for a user who passes uncorrected production rates. The finding illustrates why the stored outputs of a model are a better test than its published equations.

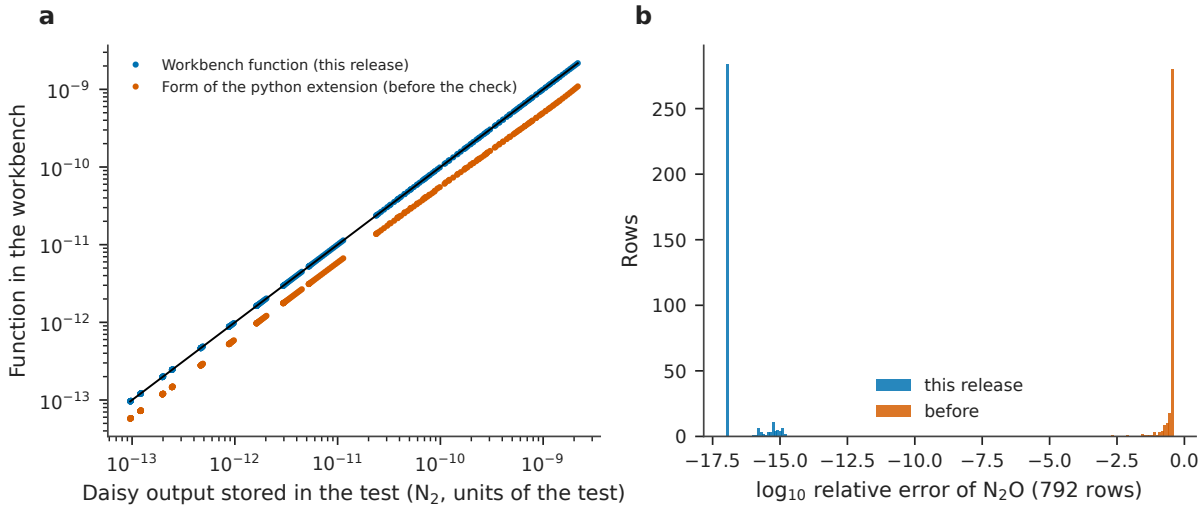


Figure 7: Denitrification function against the 792 stored Daisy outputs. (a) N_2 loss; the 1:1 line is black. (b) Histogram of the relative error of the N_2O loss before and after the correction.

7.4 Calibration

Starting from a loss of 0.20, the three optimisers reached a median loss of 6×10^{-4} (sequential, 16 evaluations), 4×10^{-6} (evolution strategy, 154) and 8×10^{-8} (Bayesian, 150) in the logarithm of the flux. They recovered α_F to 0.03, 0.001 and 0.0002 decades (median over five seeds), but they did not recover K_d or α: the median errors are 0.45 to 0.54 decades for K_d and 0.48 to 0.68 decades for α. The 792 rows span a narrow range of suction (-112 to -94 cm) and a nearly constant nitrate content, so that probably many combinations of the two parameters give almost the same outputs. The run times are 4 ms (sequential), 37 ms (evolution strategy) and 1.7 s (Bayesian). The tool therefore does what an optimiser can do and shows, in the history of the search, the combinations that are equally good; it cannot make a parameter identifiable that the outputs do not constrain.

7.5 The planner

The offline reader returned the factors, station and soil that were expected for all 8 of 8 test questions (Table 5; the catalogue holds 54 factors, 10 soils and 8 stations). The guard of the online route removed the unknown factor and noted it, limited the rain scale from 0.1, 1 and 9 to 0.4, 1 and 1.6, the range of the factor, and removed the unknown station, soil and response. The online assistant itself was not run (no key and no network in the environment), so nothing is claimed about its answers, only about what the guard does to an answer.

Table 5: Test questions of the planner and the factors that the offline reader returned (all of the expected factors, station and soil were present in every case).

Question	Factors returned	Expected
How does the irrigation trigger of -100 to -800 cm change drainage on a sandy loam in Bologna over 10 years?	irrig, trig	yes
What happens to nitrate leaching if I cut the nitrogen dose by 20 % in Foulum?	nfac, mm	yes
Compare 20 % less rain and 20 % more rain at Jyndevad on a loamy sand	precip	yes
Warming of 2 degrees and 4 degrees in Poland, Jadwisin	temp	yes
Is drip irrigation better than rainfed on a silt loam in the Czech Republic?	irrig, method	yes
Effect of the sensor depth 10, 30 and 60 cm on the irrigation trigger	irrig, trig, z	yes
Pesticide half-life of 20 and 60 days and sorption koc of 50 and 200 l/kg	koc, half	yes
What if the rooting depth is 60 and 120 cm in Bratislava?	root	yes

8. Results of the bridge tests

8.1 Parsing and summarising

The six logs were read with relative differences between the two readers of at most 3×10^{-15} (largest log) and 0 for the calibration log, which is a single row. The seasonal nitrogen summaries differ from the log columns by at most 0.049 kg ha^{-1} in the uptake, 0.049 in the mineralisation, 0.005 in the denitrification and the leaching, and 0 in the fertiliser; these differences are the rounding of the summary to 0.1 (nitrogen) or 0.01 kg ha^{-1}, and the Daisy error column is 0 in all 20 seasons. In the chemical budget the residual of Eq. (2) is 0.12 kg ha^{-1} or less in each of ten seasons and 0.47 over the ten, 0.02 % of the 2 290 kg ha^{-1} applied, which are the rounding of the log values. The terms of the budget can be trusted to the precision of the log.

The two nitrogen runs show what the layers are for. Their mean fertiliser is the same (229 kg N ha^{-1} per season), the crop takes up 341 and 339 kg N ha^{-1}, but the leaching below 1 m is 4.4 in run 3b and 17.0 in run 4b, a factor of 3.9, and the denitrification is 15.8 and 14.5 (Table 6). A person who compared only the uptake would see no difference. The bridge reports all terms of the budget side by side, and the nitrogen layer shows them season by season.

Table 6: Mean seasonal nitrogen budget of the two bundled annual nitrogen logs as reported by the bridge (kg N ha^{-1}, ten seasons).

Run	Fertiliser	Mineralisation	Deposition	Crop uptake	Denitrification	Leaching
3b	229.0	136.0	13.2	341.5	15.8	4.4
4b	229.0	134.9	13.2	338.5	14.5	17.0

8.2 The denitrification defect

The first comparison of the shared denitrification function with the 792 stored outputs gave a median relative error of 40 %, because the rescaling of the carbon dioxide production for the pressure head was applied to inputs that Daisy had already rescaled. After the correction the largest relative difference is 2.9×10^{-15} (N_2) and 1.7×10^{-15} (N_2O) (workbench checks). The function is used in the workbench to draw the response curves and, through the bridge, in the reading of the gaseous nitrogen losses.

8.3 Three routes from Daisy to a decision

The study ran 10 190 simulations in 17 experiments in 17.5 days of Daisy time (mean 148 s, median 145 s per run). Evaluating Engine 1 for 10^5 scenarios takes about 2 s (Paper 2), that is 2×10^{-5} s per scenario, a factor of 7×10^6 against the median Daisy run. The compact experiments of the workbench stand in between, 368 runs in about 15 hours (workbench checks). The three routes are not alternatives of one kind but answer different questions (Table 7, Fig. 8).

Table 7: The three routes from Daisy to a decision.

	Ensemble and Engine 1	Compact experiment (workbench)	Own live run
Soil and weather	ten soils, one weather record, the design of the study	as chosen in the workbench (library or own soil, bundled or own weather)	own soil table and weather
Time	microseconds per scenario	about 0.2 to 4 hours per experiment	minutes per run
Needs Daisy	no	yes (Windows)	yes (Windows)
Answers	ranking of options, response curves, regime maps within the design	the shape of a response for the chosen conditions	the budget of one set-up, for the layers of the suite
Main limit	fixed design; offset of absolute levels (Paper 2)	coarse levels (workbench)	one run, one set-up

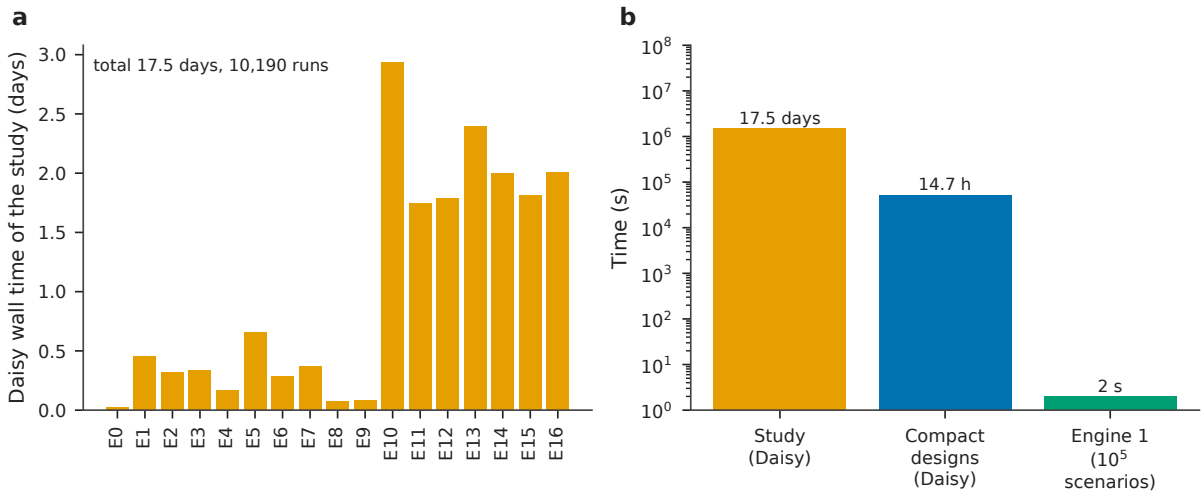


Figure 8: (a) Daisy wall-clock time of the 17 experiments of the study (total 17.5 days). (b) Time of the study, of the compact designs of the workbench and of Engine 1 for 10^5 scenarios (logarithmic scale).

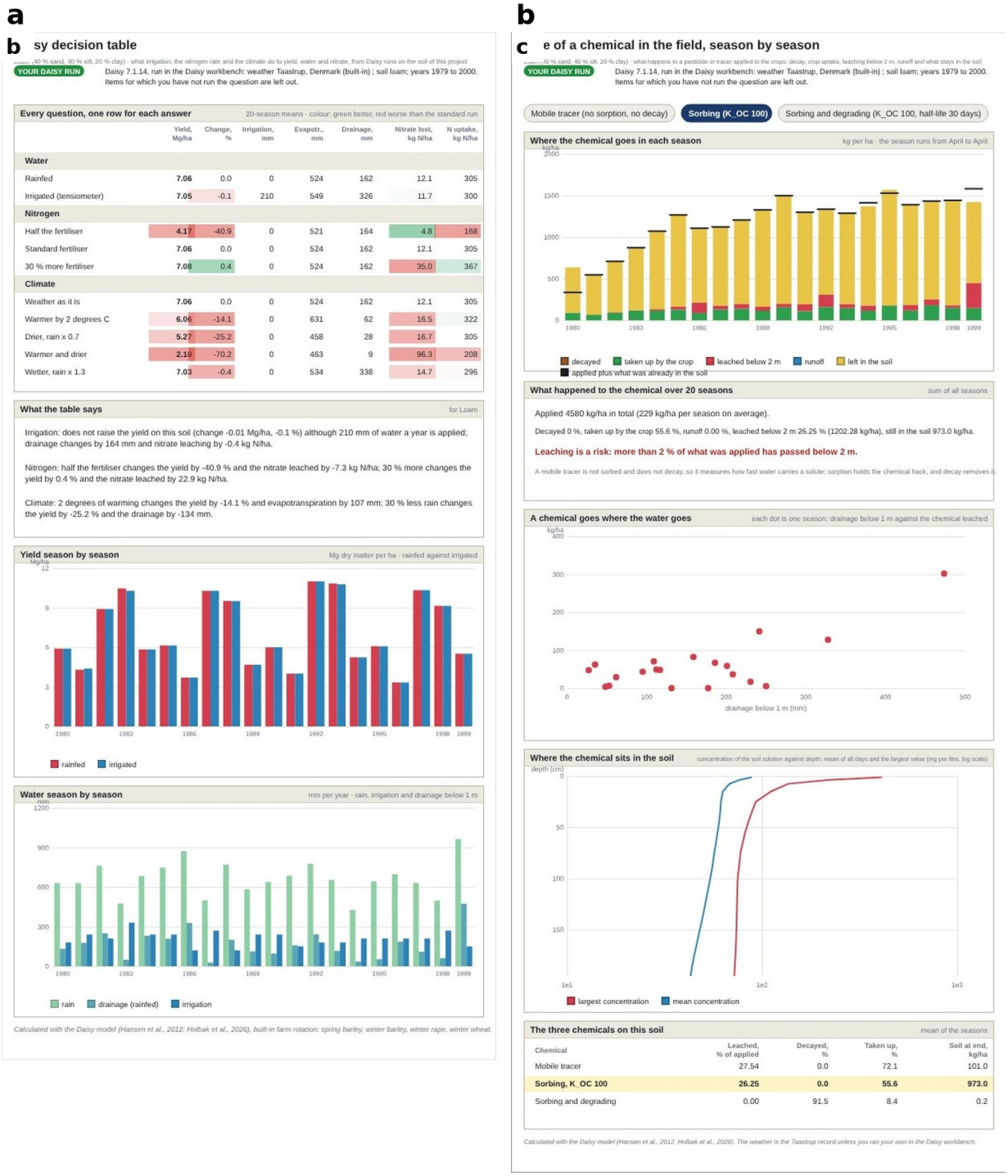


Figure 9: Results of the hand-over. (a) The Daisy decision table with the labels of the run (rainfed and irrigated, half and 30 % more fertiliser, warmer and drier climates). (b) The fate of a chemical for a mobile, a sorbing and a sorbing and degrading tracer.

8.4 Agreement of the fast route with Daisy

The agreement of Engine 1 with Daisy was tested on 1 000 runs of the sensor, trigger and rain experiment (E12, coarse loam) that were not used to build its lookups: the ranking of the trigger,

sensor-depth and rainfall options is preserved (Spearman 0.64 to 0.97) and the absolute level has a diagnosed offset (Paper 2). This is the evidence that the first route can be used to choose between options. It is not evidence that the second and third routes agree with the first for a soil outside the study; for those the user is told, in the header of the layer, which weather and soil were used.

9. Discussion

What the workbench changes. A user can ask a question, see the runs that would answer it, run them and read the answer next to the study, without writing a file. The structure of the workbench is the structure of the work: data, question, results, decision. For the developer, the set-up editor, the process map and the calibration tool remain in reach.

What the checks say about trust. The checks are of three kinds. Those that compare the workbench with an independent computation (the Richards reference, the parsers of the logs of the bridge) give agreement to the precision of the method. The check against the stored outputs of Daisy found a real defect, which would have given 40 % too little denitrification to every user of the curve tool, and this was only visible because the stored outputs were used and not the equations. The check of the optimisers shows a property of the problem and not of the tool. The cheapest way to find the defects of a graphical tool is to compare each of its parts with something that was not made with it.

Limits. (i) Daisy could not be run in the environment used for the checks, so the agreement of the *runs* of the workbench with the study was not tested; what was tested is the design, the parsing, the solver and the functions. (ii) The compact designs lose a median of 13 % of the shape of a marginal curve. (iii) Three experiments are not offered as designs. (iv) The soil column is slow and not converged for a ponded fine soil. (v) Daisy is bundled for Windows only. (vi) No field measurements are used; the workbench shows what the model says and does not say that the model is right for a field. (vii) The calibration of a model on its own outputs cannot show that the calibration works on measurements.

Why the rule of origin matters. A bridge between a model and a tool has two kinds of risk. One is translation (a column read wrongly, a unit lost); the tests above address it. The other is substitution: a layer that shows a number from a different source under the name of the model. The rule that a Daisy layer shows a Daisy number or nothing removes the second kind by design, and the label

of the run keeps it visible.

What the tests did not cover. (i) Daisy was not run in this study, so the whole chain from a set-up to a filled layer was tested in parts and not as one. The parts are the places where a defect can hide, and the one defect found (the denitrification function) was in a part. An end-to-end test with a Daisy installation is the first thing to do. (ii) The bundled logs have ten seasons of one rotation; the logs of a year with a different crop, a long fallow or irrigation have columns that were not exercised. (iii) The two coarse soils of the study and own soil tables are not mapped, so the layers of the suite keep the soil of the project in those cases. (iv) The agreement of the first route with Daisy was tested against Daisy runs of the study itself, not against measurements. (v) The summariser reduces a run to seasons; a question that depends on the day (the date of the first irrigation) needs the daily logs, which the workbench plots but the layers do not read.

Generalisation. The design is not specific to Daisy: a parser of the logs of another model, a summariser to seasonal terms, a map of the soils, a map of questions to layers and one shared object are the parts of any such bridge. The rule of origin and the tests against the files of the model's own project are what make it checkable.

10. Conclusions

The workbench makes the Daisy model usable through a question-first path in which a request in plain words becomes a design that is checked against what Daisy can vary, makes the experiments of a large published ensemble repeatable in about an hour each instead of days, and lets a user check the model at the level of its processes, a soil column and a calibration. The bridge hands the runs to decision layers under one rule, that a Daisy layer shows a Daisy result only from a Daisy run, reads the logs exactly, closes the chemical budget to 0.02 % of the amount applied, and offers three routes from Daisy to a decision that differ by seven orders of magnitude in time. Each computing part was verified against an independent reference or against stored Daisy output, and the verification found and removed one defect, that of the shared denitrification function (median error 40 %). The workbench is a way to use Daisy and not a replacement for it. The next tests are an end-to-end run on a machine with Daisy, the online assistant under its guard, and a comparison of the live layers and of the runs of the workbench with field measurements.

Software, data and code availability

Software. SWAFL Desktop (version 1.1.0) is open source; the Software source is in the project repository (https://github.com/halilunaziru73-creator/Coupled-Water-Fluxes-Pore-Scale-Dynamics-Precision-Irrigation-Mulching-Feedback-Loops/tree/main/Software) and the program for Windows is SWAFL.exe (https://github.com/halilunaziru73-creator/Coupled-Water-Fluxes-Pore-Scale-Dynamics-Precision-Irrigation-Mulching-Feedback-Loops/releases/latest/download/SWAFL.exe). The whole workbench also runs in a browser, with nothing to install (link in the repository README).

Manual. The Manual (https://github.com/halilunaziru73-creator/Coupled-Water-Fluxes-Pore-Scale-Dynamics-Precision-Irrigation-Mulching-Feedback-Loops/raw/main/Software/v1.1.0/SWAFL_Desktop/docs/manual_v1.1.0/SWAFL_Manual_v1.1.0.pdf) documents every control, screen and equation of the release, with labelled screenshots.

Data. The complete Daisy ensemble outputs (all 10 190 runs, raw and derived) are archived on Zenodo, doi: 10.5281/zenodo.23043626; the run table and a summary of every experiment are in the Data folder (https://github.com/halilunaziru73-creator/Coupled-Water-Fluxes-Pore-Scale-Dynamics-Precision-Irrigation-Mulching-Feedback-Loops/tree/main/Data).

Code. Every check reported here has a script in `Manuscripts/reproducibility/p456` (Node.js scripts that load the shipped code or drive the shipped application, Python scripts for the analysis of the ensemble and the figures); `README_REPRODUCE.md` lists the steps. Daisy itself could not be run in the environment used for these checks, no field measurements are used, and the screenshots are archived rather than regenerated.

Author contributions

N.H. designed the workbench, wrote the software and the checks, analysed the data and wrote the article.

Funding

No funding was received for this work.

Competing interests

The author declares no competing interests.

Acknowledgements

The Daisy model is developed and maintained at the University of Copenhagen. The author thanks the maintainers of the open-source libraries cited.

References

Per Abrahamsen and Søren Hansen. Daisy: an open soil-crop-atmosphere system model. *Environmental Modelling and Software*, 15:313–330, 2000.

Michael A. Celia, Efthimios T. Bouloutas, and Rebecca L. Zarba. A general mass-conservative numerical solution for the unsaturated flow equation. *Water Resources Research*, 26:1483–1496, 1990. doi: 10.1029/WR026i007p01483.

Reinder A. Feddes, Piotr J. Kowalik, and Henryk Zaradny. *Simulation of Field Water Use and Crop Yield*. Pudoc, Wageningen, 1978.

Naziru Halilu. Coupled water fluxes and pore scale dynamics under joint precision irrigation and evaporation control mulching: ensemble of 10 190 Daisy simulations. Zenodo dataset, 2026.

Nikolaus Hansen and Andreas Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9:159–195, 2001.

Søren Hansen, Hans E. Jensen, Niels E. Nielsen, and Henrik Svendsen. DAISY: Soil plant atmosphere system model. Technical report, The Royal Veterinary and Agricultural University, Copenhagen; NPO Report A10, Danish Environmental Protection Agency, Copenhagen, 1990.

Søren Hansen, Per Abrahamsen, Christian T. Petersen, and Merete Styczen. Daisy: model use, calibration, and validation. *Transactions of the ASABE*, 55:1317–1335, 2012. doi: 10.13031/2013.42244.

Maja Holbak, Per Abrahamsen, Celine R. Grønning, Tobias Klöffel, Magdalena V. Schwartzkopff, Merete E. Styczen, Simon F. Svane, and Efstathios Diamantopoulos. The Daisy model: a

tool for simulating agricultural systems — recent (2012–2025) developments, calibrations and validations. *Agricultural Systems*, 238:104846, 2026. doi: 10.1016/j.agsy.2026.104846.

James W. Jones, Gerrit Hoogenboom, Cheryl H. Porter, Kenneth J. Boote, William D. Batchelor, L. A. Hunt, et al. The DSSAT cropping system model. *European Journal of Agronomy*, 18: 235–265, 2003. doi: 10.1016/S1161-0301(02)00107-7.

Brian A. Keating, Peter S. Carberry, Graeme L. Hammer, Merv E. Probert, Michael J. Robertson, Dean Holzworth, et al. An overview of APSIM, a model designed for farming systems simulation. *European Journal of Agronomy*, 18:267–288, 2003. doi: 10.1016/S1161-0301(02)00108-9.

Marc C. Kennedy and Anthony O'Hagan. Bayesian calibration of computer models. *Journal of the Royal Statistical Society: Series B*, 63:425–464, 2001. doi: 10.1111/1467-9868.00294.

Jonas Mockus, Vytautas Tiesis, and Antanas Zilinskas. The application of Bayesian methods for seeking the extremum. *Towards Global Optimization*, 2:117–129, 1978.

Yechezkel Mualem. A new model for predicting the hydraulic conductivity of unsaturated porous media. *Water Resources Research*, 12:513–522, 1976. doi: 10.1029/WR012i003p00513.

Saman Razavi, Bryan A. Tolson, and Donald H. Burn. Review of surrogate modeling in water resources. *Water Resources Research*, 48:W07401, 2012. doi: 10.1029/2011WR011527.

Jerome Sacks, William J. Welch, Toby J. Mitchell, and Henry P. Wynn. Design and analysis of computer experiments. *Statistical Science*, 4:409–423, 1989. doi: 10.1214/ss/1177012413.

Jiří Šimůnek, Martinus Th. van Genuchten, and Miroslav Šejna. Development and applications of the HYDRUS and STANMOD software packages and related codes. *Vadose Zone Journal*, 7:587–600, 2008. doi: 10.2136/vzj2007.0077.

Jiří Šimůnek, Martinus Th. van Genuchten, and Miroslav Šejna. Recent developments and applications of the HYDRUS computer software packages. *Vadose Zone Journal*, 15(7), 2016. doi: 10.2136/vzj2016.04.0033.

Pasquale Steduto, Theodore C. Hsiao, Dirk Raes, and Elias Fereres. AquaCrop—the FAO crop model to simulate yield response to water: I. concepts and underlying principles. *Agronomy Journal*, 101:426–437, 2009. doi: 10.2134/agronj2008.0139s.

Martinus Th. van Genuchten. A closed-form equation for predicting the hydraulic conductivity of unsaturated soils. *Soil Science Society of America Journal*, 44:892–898, 1980. doi: 10.2136/sssaj1980.03615995004400050002x.

A. Checks and scripts

Table 8: Check to script map (scripts in `Manuscripts/reproducibility/p456`).

Check	What is compared	Script → output
Design sizes (Table 3)	presets against the ensemble	`p4_design_sizes.js` → `out_p4_design.json`
Fidelity and run time (Fig. 5)	interpolated against full marginals; `run_seconds`	`p4_design_fidelity.py` → `out_p4_fidelity.csv`, `out_p4_runtime.csv`
Soil column (Table 4, Fig. 6)	workbench against scipy BDF	`p4_richards.js`, `p4_richards_ref.py`
Denitrification (Fig. 7)	792 stored outputs	`p5_bridge_checks.js`
Calibration	recovery of three parameters	`p4_calibration.js`
Figures and tables		`p456_figs_schematics.py`, `p456_figs_data.py`, `p456_tables.py`

B. Checks and scripts

Table 9: Check to script map (scripts in `Manuscripts/reproducibility/p456`).

Check	What is compared	Script → output
Parsers and nitrogen summaries	bridge against `pandas`	`p5_bridge_checks.js`, `p5_bridge_checks.py` → `out_p5_bridge.json`
Chemical budget	residual of Eq. (2)	`p5_chem_closure.js` → `out_p5_chem.json`
Denitrification	792 stored outputs	`p5_bridge_checks.js` → `out_p5_bridge_js.json`
Run time	`run_seconds` of the ensemble	`p4_design_fidelity.py` → `out_p4_runtime.csv`
Empty state of the Daisy layers	application opened afresh	`p5_empty_state.js`
Figures		`p456_figs_schematics.py`, `p456_figs_data.py`

C. Checks of the planner

The test of the planner is `Manuscripts/reproducibility/p456/p7_planner_checks.js`, with the output `out_p7_planner.json`.